Course Outline: Telemetry Best Practices (Guardrails)

Title: Telemetry Best Practices (Guardrails) **Skill:** 43 — Collecting telemetry and context-related info from the user and your application **Learnpack:** + Mejores prácticas en telemetría (Guardrails) **File:** s43-w15d43-mejores-practicas-en-telemetria-guardrai **Prerequisite:** Capturing Telemetry in Next.js (Skill 43 — the capture hooks implementation) **Target Audience:** Beginner developers in a full-stack AI bootcamp completing a production-grade telemetry pipeline **Total Lessons:** 9 **Format:** Mixed — 6 📖 + 1 💻 + 1 🤯 + 1 outro (11% hands-on)

Course Description

This course completes the telemetry pipeline by adding the safety layer that makes it production-ready. Students implement 5 guardrails that protect user privacy, prevent data leaks, and maintain signal quality: consent-first enforcement, free-form text scrubbing, property allowlists, sampling, and environment separation. The capstone is a `guardEvent()` function that applies all guardrails in sequence before any event reaches the sender.

Course Philosophy

This course emphasizes:

- Guardrails are not optional: a telemetry system without consent enforcement, scrubbing, or allowlists is a privacy and compliance liability
- One function to rule them all: `guardEvent()` concentrates all safety logic in a single, testable function
- Composable and testable: each guardrail is a pure function that can be tested independently

---

Course Statistics Summary

Section	Lessons
00 - Welcome	1
01 - The 5 Guardrails	5
02 - Assessment	2
03 - Outro	1
Total	**9**

00.0 Welcome 📖

Learning Objectives:

- Understand what guardrails are and why they are the mandatory final step in a telemetry pipeline
- Connect this course to the full telemetry pipeline: taxonomy → sender → capture hooks → guardrails
- Preview the 5 guardrails that this course implements

Content Outline:

- The problem this course solves: without guardrails, the telemetry system built in the previous courses could:
  - Send analytics events for users who haven't consented (GDPR/CCPA violation)
  - Accidentally include PII in telemetry events (privacy liability)
  - Leak internal property names to external analytics systems (security risk)
  - Overwhelm the backend with verbose log events (operational cost)
  - Send dev/staging events that corrupt production analytics (data quality issue)
- The 5 guardrails: consent check, scrubbing, allowlist, sampling, environment separation
- The capstone: `guardEvent()` — a single function that applies all 5 guardrails before any event is sent

Transitions: You've built the sender, the capture hooks, and the event taxonomy. This course adds the final layer that makes the system compliant, safe, and cost-efficient.

Key Principles:

- Guardrails fail safe: if consent is uncertain, don't send; if a property is not in the allowlist, strip it
- Test guardrails with real event objects, not mock strings
- The order of guardrails matters: consent check first (cheapest), scrubbing and allowlist next, sampling last

Examples:

- Without consent enforcement, a developer who adds `track()` to a public landing page sends analytics for every visitor regardless of their cookie consent preferences — a GDPR violation before the user even sees a banner
- Without scrubbing, a `search_performed` event that includes `query: userInput` could contain a user's full name, address, or password if they accidentally typed it in the search box

---

01.0 Telemetry Guardrails 🕮

Learning Objectives:

- Name all 5 telemetry guardrails and explain what each one prevents
- Understand the order in which guardrails should be applied
- Explain how `guardEvent()` acts as a pipeline gate before the sender

Content Outline:

- The 5 guardrails:

1. **Consent check**: Does the user consent to analytics? If `consent.analytics=false`, return null — no event sent
2. **Scrubbing**: Does the event contain free-form text fields (comments, search queries, feedback)? Strip or truncate them to prevent accidental PII leakage
3. **Allowlist**: Does every property in the event exist in the approved allowlist? Strip any properties not in the list — prevents accidental leakage of internal state
4. **Sampling**: Is this a verbose/high-frequency event (e.g., performance logs, scroll events)? Apply a sampling rate (e.g., only send 10% of them) to reduce volume without losing statistical signal
5. **Environment separation**: Is this a dev or staging environment? Gate verbose logging or drop events entirely in non-production environments

- The `guardEvent()` function signature:

```
function guardEvent(event, options) {
  // Returns the (possibly modified) event if it should be sent
  // Returns null if the event should be dropped
}
```

- Why order matters:
  - Consent check is first: no point scrubbing or sampling an event that won't be sent
  - Allowlist is after scrubbing: no point allowlisting properties that will be scrubbed anyway
  - Sampling is last: only sample events that pass all other checks (so your sample is always of valid, compliant events)
- Integration: `guardEvent()` sits between capture and the sender's `track()` function

Transitions: The overview is complete. Let's implement each guardrail, starting with the most legally critical.

Key Principles:

- `guardEvent()` has a single contract: receive an event, return the event (possibly modified) or null
- Every guardrail is stateless: `guardEvent()` does not need access to application state beyond what's passed to it
- Test `guardEvent()` by calling it with different events and checking the output — no mocking required

Examples:

- A team that deploys telemetry without consent enforcement receives a CNIL fine (French data protection authority) for sending analytics without consent — this happened to major companies
- An allowlist of 20 properties protects against the case where a developer accidentally passes the entire user object as event properties, leaking email, hashed password, and session tokens

---

01.1 Consent and Scrubbing 🕮

Learning Objectives:

- Implement a consent-first check that blocks all analytics events when consent.analytics=false
- Implement a field scrubber that sanitizes free-form text fields in event properties
- Explain the legal requirement behind consent-first telemetry and the risk of free-form text

Content Outline:

- The consent check:
  - Where consent data lives: typically in application state or a cookie (e.g., `consent.analytics: boolean`)
  - The rule: if `consent.analytics=false`, return null — no event is sent, regardless of type
  - Don't check consent inside individual `track()` calls — check it once in `guardEvent()` where it's impossible to forget
  - GDPR/CCPA requirement: analytics tracking requires explicit opt-in (in EU) or opt-out (in US/CA); telemetry is analytics
  - Functional cookies (strictly necessary for the app to work) are typically exempt — but behavioral analytics are not
- The scrubbing guard:
  - The risk: developers call `track('search_performed', { query: event.target.value })` without realizing the user might type sensitive data into a search box
  - The scrubbing approach: identify fields in the event that contain user-input text (typically `query`, `comment`, `message`, `feedback`, `description`) and either:
    - Truncate to 100 characters (reduces but doesn't eliminate risk)
    - Apply a scrubber function that replaces potential PII patterns (emails, phone numbers) with placeholders
    - Simply remove the field entirely if the query value isn't needed for the event's purpose
  - The allowlist approach (more robust): instead of trying to scrub, define EXACTLY which properties are allowed (covered in the next lesson)
  - At a minimum: never pass `event.target.value` directly as a property value without sanitization

Transitions: Consent and scrubbing protect against accidental data collection. The next guardrail provides a positive enforcement model: only send exactly what's in the approved list.

Key Principles:

- Consent check should be the first thing `guardEvent()` does — it's the cheapest operation and the most legally important
- Scrubbing is defensive programming: assume developers will accidentally put PII in free-form text fields
- A field scrubber that removes emails (`/[^\s@]+@[^\s@]+\.[^\s@]+/g`) and phone numbers catches the most common accidental PII leaks

Examples:

- A user searching for "John Smith 555-867-5309 account reset" accidentally sends a full PII event without scrubbing — then someone files a GDPR erasure request and you have to find this event in your analytics system

- A feedback field that sends `{ feedback: "I can't log in with john@example.com" }` is a GDPR violation even though it was captured with good intent

---

01.2 Property Allowlists 🕮

Learning Objectives:

- Implement a property allowlist that strips any event property not in an approved list
- Explain the security benefit of allowlisting over blocklisting for telemetry properties
- Design a minimal allowlist for a common telemetry event

Content Outline:

- The allowlist concept:
  - A blocklist (what NOT to send) grows over time and is always incomplete — there's always one more property you forgot to block
  - An allowlist (ONLY these properties are allowed) is complete by definition — anything not in the list is dropped
  - Allowlisting is a "default deny" security posture, which is safer for telemetry
- Implementing the allowlist:
  - Define a list of allowed top-level property keys per event type (or a universal base allowlist)
  - When processing an event, create a new properties object that only includes keys from the allowlist
  - Example universal base allowlist: `['event', 'timestamp', 'userId', 'sessionId', 'appVersion', 'env', 'route']`
  - Example per-event allowlist: `search_performed` allows `['query', 'result_count']`
- The stricter approach: per-event allowlists
  - Universal allowlists are better than nothing but don't prevent event-specific leaks
  - Per-event allowlists: a map from event name to allowed property keys
  - Any property not in the map for that event type is stripped
- Handling nested objects:
  - Allowlists work on top-level properties; nested objects should be flattened or avoided entirely in telemetry
  - Exception: `context` and `user` objects in the base event schema are pre-defined and safe

Transitions: Allowlisting ensures you only send what you intend to send. Sampling addresses a different problem: what if the volume of compliant events is still too high?

Key Principles:

- Allowlists should be reviewed by a privacy officer or security team before deployment
- A failed allowlist check should strip the property, not drop the event — the event itself may still be valuable
- Allowlists evolve with the product: add properties deliberately, never just pass the entire object

Examples:

- A developer builds a checkout flow and passes `track('checkout_submitted', { ...checkoutState })` — without an allowlist, `checkoutState` might contain `cardLastFour`, `billingAddress`, and `email`
- With an allowlist of `['checkout_id', 'item_count', 'checkout_total', 'payment_method_type']`, the same call is safe

01.3 Sampling and Environment Separation 🕮

Learning Objectives:

- Implement a sampling function that sends only a configurable percentage of verbose events
- Implement an environment gate that blocks or modifies events in dev/staging environments
- Explain when to use sampling and how it maintains statistical validity

Content Outline:

- Sampling:
  - The problem: high-frequency events (API response times, scroll depth, mousemove) generate enormous event volumes that are expensive to store and process
  - The solution: sample — send only X% of these events, chosen randomly
  - Statistical validity: with 10% sampling, 100 events become 10, but the distribution of values remains representative of the full dataset
  - Implementation: `if (event.event in VERBOSE_EVENTS && Math.random() > SAMPLE_RATE) return null`
  - Sampling rate guidance:
    - Performance events (API latency, Web Vitals): 10% or lower
    - Error events: 100% (never sample errors — you need all of them)
    - Key user actions (purchases, enrollments): 100% (never sample high-value conversions)
    - Verbose informational events (scroll depth, hover): 5-10%
  - The sampling metadata: add `sampled: true` and `sample_rate: 0.1` to sampled events so analysts can weight them correctly
- Environment separation:
  - The problem: dev and staging events mixed into production analytics corrupt every metric and dashboard
  - The solution: check `env` in the event context; in dev/staging, either drop the event or send it to a separate dev endpoint
  - Recommended pattern: in dev, `console.log` the event instead of sending it (allows developer to see events without polluting production data)
  - In staging: send to a separate telemetry project or bucket with a `staging: true` flag
  - Never let staging events reach production analytics
  - The `env` field in the event context (set during execution context capture) is the source of truth

Transitions: All 5 guardrails are now understood. The next lesson implements them all in a single `guardEvent()` function.

Key Principles:

- Always sample verbose events, never key conversions — the cost of sampling a conversion is losing visibility into a business outcome
- Sampling in `guardEvent()` means the sender never sees sampled-out events — the buffer stays clean
- Environment separation is a hard gate, not a soft filter — no `console.warn` in production, no staging events in production analytics

Examples:

- API latency events at 100% would generate ~50,000 events per day per 1,000 users; at 10% sampling, that's 5,000 — a 90% cost reduction with full statistical validity
- A developer testing checkout leaves behind 200 fake purchase events in staging; without environment separation, the revenue metrics for that day show a $20,000 spike that never happened

---

01.4 Build a Telemetry Guard 💻

Challenge Prompt: Build a `guardEvent(event, options)` function that applies all 5 guardrails in sequence. The function receives an event object and an options object containing `{ consent, allowlist, sampleRate, env }`. It returns the sanitized event if it should be sent, or null if it should be dropped.

Solution Code:

```
const SCRUB_FIELDS = ['query', 'comment', 'message', 'feedback',
'description'];
const EMAIL_REGEX = /[^\s@]+@[^\s@]+\.[^\s@]+/g;
const PHONE_REGEX = /\b\d{3}[-.]?\d{3}[-.]?\d{4}\b/g;

function scrubField(value) {
  if (typeof value !== 'string') return value;
  return value
    .replace(EMAIL_REGEX, '[email]')
    .replace(PHONE_REGEX, '[phone]')
    .substring(0, 200);
}

function guardEvent(event, options = {}) {
  const {
    consent = {},
    allowlist = null,
    sampleRate = 1.0,
    env = 'production',
  } = options;

  // 1. Consent check
  if (consent.analytics === false) {
```

```
    return null;
  }

  // 2. Scrubbing
  const scrubbedProperties = {};
  for (const [key, value] of Object.entries(event.properties || {})) {
    scrubbedProperties[key] = SCRUB_FIELDS.includes(key)
      ? scrubField(value)
      : value;
  }

  // 3. Allowlist filter
  const filteredProperties = allowlist
    ? Object.fromEntries(
        Object.entries(scrubbedProperties).filter(([key]) =>
allowlist.includes(key))
      )
    : scrubbedProperties;

  // 4. Sampling
  if (sampleRate < 1.0 && Math.random() > sampleRate) {
    return null;
  }

  // 5. Environment gate
  if (env !== 'production') {
    console.log('[telemetry dev]', event.event, filteredProperties);
    return null;
  }

  return {
    ...event,
    properties: filteredProperties,
  };
}
```

Challenge Code:

```
const SCRUB_FIELDS = ['query', 'comment', 'message', 'feedback',
'description'];
const EMAIL_REGEX = /[^\s@]+@[^\s@]+\.[^\s@]+/g;
const PHONE_REGEX = /\b\d{3}[-.]?\d{3}[-.]?\d{4}\b/g;

function scrubField(value) {
  // Scrub emails and phone numbers, truncate to 200 chars
  // your code here
}

function guardEvent(event, options = {}) {
  const {
    consent = {},
```

```
    allowlist = null,
    sampleRate = 1.0,
    env = 'production',
  } = options;

  // 1. Consent check — return null if consent.analytics is false
  // your code here

  // 2. Scrubbing — scrub sensitive fields in event.properties
  // your code here

  // 3. Allowlist filter — strip properties not in the allowlist
  // your code here

  // 4. Sampling — drop event based on sampleRate
  // your code here

  // 5. Environment gate — in non-production, console.log and return null
  // your code here

  return {
    ...event,
    properties: filteredProperties,
  };
}
```

02.0 Guardrails in Practice 🕮

Learning Objectives:

- Integrate `guardEvent()` into the telemetry pipeline built across this course sequence
- Identify where in the code `guardEvent()` should be called and how it interacts with the sender
- Describe what a production-ready telemetry pipeline looks like end-to-end

Content Outline:

- Integrating `guardEvent()` with the sender:
  - The sender's `track()` function should call `guardEvent()` before adding the event to the buffer
  - Pattern: `const safe = guardEvent(event, options); if (safe !== null) buffer.push(safe);`
  - This ensures the sender never sees unchecked events — `guardEvent()` is the single gate
- The complete telemetry pipeline (built across this course sequence):
  1. **Taxonomy** (previous course sequence): defined what signals to collect and why
  2. **Sender** (Sending Telemetry course): batch + debounce + sendBeacon
  3. **Capture hooks** (Capturing Telemetry in Next.js course): routeChangeComplete, user action handlers, error handlers, Web Vitals
  4. **Guardrails** (this course): `guardEvent()` with consent, scrubbing, allowlist, sampling, env
- What's still needed in production:

    - Backend ingestion: an endpoint that receives batches and routes them to your analytics system
    - Data warehouse: storage for analysis (BigQuery, Snowflake, or similar)
    - Dashboards: visualizations on top of the stored data
    - But none of that matters if the frontend pipeline sends bad data — guardrails are the quality gate
- Testing the complete pipeline:
    - Unit test each guardrail independently
    - Integration test `guardEvent()` with a sample event that fails each guardrail
    - End-to-end test: verify that a `checkout_submitted` event with a PII-containing query field reaches your analytics system without the PII

Transitions: Test your understanding of the complete guardrails implementation.

Key Principles:

- `guardEvent()` should be called in exactly one place (the sender's `track()` function) — never sprinkled across event handlers
- A telemetry system is only as good as its worst guardrail: one missing consent check undoes all the others
- The sender should be oblivious to guardrails — it just sends what it receives; `guardEvent()` is the filter before the sender

Examples:

- A team that adds `guardEvent()` to their pipeline mid-flight discovers that 30% of their search events contained email addresses that were being sent to their analytics vendor — the allowlist caught this retroactively
- A team that tests their consent guard discovers that their cookie banner was incorrectly setting `consent.analytics=true` before the user made a choice — they would never have found this without testing `guardEvent()` explicitly

---

02.1 Telemetry Guardrails 🧠

Learning Objectives:

- Identify which guardrail applies to a described scenario
- Predict the output of `guardEvent()` for a given event and options
- Diagnose missing guardrails from a described telemetry implementation

Question Format: Scenario-based

Questions:

1. A user with `consent.analytics=false` triggers a `checkout_submitted` event. Your telemetry system sends the event anyway. Which guardrail is missing, and where in `guardEvent()` should it be added?

2. A `search_performed` event arrives at `guardEvent()` with `properties: { query: "John Smith john@example.com", result_count: 42 }`. After scrubbing, what does the `query` field look like? After applying an allowlist of `['query', 'result_count']`, does the event pass?

3. You have 50,000 daily active users who each trigger 3 API latency events per page load. At 100% sampling, that's 150,000 API latency events per day. If you set `sampleRate: 0.05`, how many events reach your backend per day? Is the data still statistically useful for detecting latency trends?

4. A developer runs the app in development mode and triggers a `course_enrolled` event. Your `guardEvent()` has an environment gate set to `env !== 'production' → return null`. Will this event reach your analytics backend? What will happen instead?

5. A developer adds telemetry to a user profile form: `track('profile_updated', { ...profileState })`. The `profileState` object includes `{ name, email, bio, avatarUrl, role, plan }`. Your allowlist for `profile_updated` is `['role', 'plan']`. After the allowlist guardrail, what does the event's properties object contain?

Evaluation Criteria:

- Q1: Consent check is missing; it should be the first check in `guardEvent()`; returns null if `consent.analytics=false`
- Q2: After scrubbing, `query: "John Smith [email]"` (email replaced); the event passes the allowlist check because `query` and `result_count` are in the allowlist
- Q3: 50,000 × 3 × 0.05 = 7,500 events/day; yes, 7,500 is statistically sufficient for detecting trends
- Q4: The event will NOT reach analytics; `guardEvent()` returns null and logs the event to `console.log('[telemetry dev]', ...)` instead
- Q5: After allowlist, `properties: { role: 'student', plan: 'free' }` — all other fields including email are stripped

Score Interpretation:

- 4-5 correct: Strong grasp of the guardrails and how `guardEvent()` applies them
- 2-3 correct: Good foundation; review which guardrail applies to which scenario
- 0-1 correct: Return to the 5 guardrails and their order in `guardEvent()`

03.0 Outro

Content Outline:

- Course recap: what students accomplished
  - Implemented all 5 telemetry guardrails: consent check, scrubbing, allowlist, sampling, and environment separation
  - Built `guardEvent()` — a single function that applies all guardrails before any event reaches the sender
  - Integrated `guardEvent()` into the complete telemetry pipeline
- Connection to bigger picture — the complete telemetry pipeline:
  1. **Taxonomy**: what signals to collect and why (Understanding Telemetry learnpack)

2. **Sender**: batch + debounce + sendBeacon (Sending Telemetry learnpack)
3. **Capture hooks**: Next.js router, user actions, errors, Web Vitals (Capturing Telemetry learnpack)
4. **Guardrails**: `guardEvent()` — this course
5. **Backend**: ingestion endpoint → data warehouse → analytics dashboards (Skill 44+)

- What comes next: Skill 44 (Building reports from telemetry data) uses the data that this pipeline now collects cleanly and safely
- Next steps and continued learning:
  - GDPR checklist for telemetry: iapp.org resources
  - Differential privacy: an advanced technique for preserving individual privacy while maintaining statistical signal
  - Amplitude/Segment data governance docs: how production teams manage allowlists and consent at scale
- Encouragement
  - Most production applications that collect telemetry have at least one missing guardrail. You now know all five and how to implement them. The pipeline you've built across this course sequence is ready for real users.

Note: This is conclusion only — no theory, exercises, or assessment.